

IMPROVING FINANCIAL NUMERICAL REASONING WITHOUT EXPERT DATA

Nhi Nguyen

Computer Science department
New York University
nn2536@nyu.edu

Haitong Zhu & Smruti Nalawade

Center for Data Science
New York University
{hz3916, ssn9062}@nyu.edu

ABSTRACT

Financial numerical reasoning is an essential skill for real-world financial analysis. FinQA is a common benchmark used to measure the ability of language models to perform financial numerical reasoning. Although recent domain-specialized model Fin-R1 achieves state-of-the-art performance on FinQA, it relies on costly expert-annotated chain-of-thought data while only achieving a modest 4% improvement over general-purpose models such as Qwen-2.5-Instruct. This raises an important question: *How much of this performance gain can be recovered without expensive additional training data?* In this project, we systematically evaluate three data-free approaches - Prompt engineering, Self-consistency, and Quiet self-taught reasoner - using two models, Fin-R1 and Qwen-2.5-7B-Instruct, on the FinQA dataset. Our prompt engineering results show that the gap between Fin-R1 and general-purpose models is actually smaller than reported, highlighting how thorough prompt engineering is essential for establishing fair and competitive baselines. Self-consistency yields substantial gains in general-purpose large language models (LLMs), improving accuracy by 5.8% without increasing inference time. On the other hand, Quiet self-taught reasoner requires significantly more computational power and time for training and inference than our budget allows, rendering this direction unfruitful. Together, these results demonstrate the trade-offs between data-free methods for financial numerical reasoning and highlight that strong performance gains are achievable without additional expert data. Our code can be found at <https://github.com/Dolphin2333/FinQA-with-QSTAR>.

1 INTRODUCTION

Financial numerical reasoning is a core ability required for real-world financial analysis, encompassing tasks such as financial forecasting, risk assessment, stock selection, and report analysis (Mahdavi et al., 2025; Yang et al., 2024b). Despite recent growth in large language models (LLMs), reliably performing multi-step numerical reasoning from textual and tabular financial data remains challenging (Srivastava et al., 2024; Yao et al., 2025). Financial problems often exhibit subtle domain-specific conventions, such as unit differences (e.g., revenue increase from \$540M to \$2.5B) and implicit financial assumptions (e.g., Earnings Per Share requires dividing net income by diluted weighted-average shares, not basic shares). Additionally, they require strict accuracy, as even small reasoning errors can distort downstream financial decisions. As a result, improving the numerical reasoning capabilities in LLMs is both scientifically interesting and practically consequential.

Recent domain-specialized model Fin-R1 (Liu et al., 2025) has advanced the state of the art on financial reasoning benchmarks, including FinQA (Chen et al., 2021). Fin-R1 leverages a two-stage supervised fine-tuning and reinforcement learning pipeline and a large, high-quality dataset of annotated financial Chain-of-Thought demonstrations to improve the model’s reasoning in financial settings. However, these performance gains come with significant costs, as acquiring a large high-quality Chain-of-Thought dataset requires several expert annotators capable of producing accurate multi-step reasoning traces, which can be an expensive and time-consuming process. In addition, on the FinQA benchmark, Fin-R1 achieves an accuracy of 76%, only a 4% increase compared to the best general-purpose model Qwen-2.5-32B-Instruct (72% accuracy) (Liu et al., 2025) (table 1).

Table 1: Performance as reported in Table 2 in Liu et al. (2025).

Model	Parameters	FinQA
Qwen-2.5-32B-Instruct	32B	72.0
Qwen-2.5-7BB-Instruct	7B	60.0
Fin-R1	7B	76.0

Consequently, Fin-R1’s modest improvement and dependence on costly additional data limit the application of its framework to similar domain-specific reasoning problems.

These limitations motivate our central research question: **Can we improve financial numerical reasoning in general-purpose LLMs *without* relying on additional annotated data?** A data-free approach would make progress in domain-specific reasoning problems more accessible and reproducible. Moreover, understanding how much performance can be unlocked without additional training data would offer insight into the inherent reasoning ability of general-purpose LLMs.

In this project, we investigate three data-free methods for improving financial numerical reasoning: (1) **Prompt engineering**, which guide the model toward more structured reasoning using carefully designed templates; (2) **Self-consistency** (Wang et al., 2022), and inference-time technique that samples multiple reasoning paths and aggregates them to improve accuracy and reliability; and (3) **Quiet self-taught reasoner** (Zelikman et al., 2024), a fine-tuning technique that aim to teach the models to improve its reasoning without additional data by eliciting sequences of thoughts after every single token. *Unlike Fin-R1, these methods do not require additional training data, and there has been no prior work applying them to financial numerical reasoning problems.*

We evaluate these methods on the FinQA dataset under constrained compute settings (two 80GB A100 GPUs) using two 7B models: Fin-R1 and Qwen-2.5-7B-Instruct. Our empirical analysis reveals several key findings: (1) Prompt engineering experiments show that the performance gap between Fin-R1 and general-purpose models is 50% smaller than reported by Liu et al. (2025) (section 5.1); (2) Self-consistency yields a substantial 5.8% improvements for general-purpose LLMs without increasing inference cost (section 5.2); and (3) Quiet self-taught reasoner requires more computational power and time for training markedly higher inference-time expense (section 5.3). Overall, this study clarifies the trade-offs among data-free methods for improving financial numerical reasoning and highlights how far general-purpose LLMs can be improved without relying on costly annotated data.

2 RELATED WORK

Financial numerical reasoning. Financial numerical reasoning poses challenges beyond those encountered in general natural language processing tasks, as it requires accurate multi-step arithmetic over inputs that combine text and tables and follow domain-specific conventions. FinQA (Chen et al., 2021) exemplifies this difficulty by requiring models to answer expert-written questions grounded in financial reports through multi-step arithmetic operations. Subsequent analyses show that even large pre-trained models struggle as table complexity and reasoning depth increase, particularly under unit conversions and long-range numerical dependencies (Srivastava et al., 2024; Yao et al., 2025). These findings underscore that scale alone does not resolve the precision demands of financial reasoning, where small arithmetic errors can have outsized downstream impact.

Domain-specific financial models. Recent work in financial LLMs has largely addressed reasoning challenges through either massive pretraining or heavy supervision. BloombergGPT (Wu et al., 2023) demonstrates that domain-adaptive pretraining at scale can significantly improve financial task performance, but requires proprietary data and extraordinary compute. Alternatively, Fin-R1 (Liu et al., 2025) achieves state-of-the-art FinQA accuracy using expert-annotated chain-of-thought data and reinforcement learning. Related efforts such as FinGPT (Liu et al., 2023) aim to lower barriers by releasing open-source financial models and pipelines, yet they still depend on costly annotation or tuning pipelines.

Prompting and inference-time methods. A parallel line of work explores data-free methods that improve reasoning at inference time. Chain-of-thought prompting enables LLMs to produce explicit multi-step reasoning with no parameter updates (Wei et al., 2022), including in zero-shot settings (Kojima et al., 2022). Self-consistency further improves robustness by sampling multiple reasoning paths and aggregating their outputs (Wang et al., 2022). These techniques demonstrate that much of LLM’s reasoning capability is latent and can be unlocked through careful decoding strategies. Our work extends these ideas to financial numerical reasoning, showing that prompt engineering and self-consistency can yield substantial accuracy gains for general-purpose LLMs on financial reasoning while remaining computationally inexpensive.

Self-taught reasoner. Beyond inference-time methods, self-taught reasoning approaches aim to internalize reasoning improvements into model parameters without human supervision. STaR (Zelikman et al., 2022) bootstraps training using model-generated reasoning traces, while Quiet-STaR (Zelikman et al., 2024) generalizes this idea by learning implicit “thoughts” during language modeling. Although promising, these methods incur substantial training and inference costs. While these approaches demonstrate that reasoning ability can, in principle, be learned without expert data, they shift the cost to training time and incur substantial memory and inference latency.

3 PROBLEM STATEMENT

Our objective is to improve financial numerical reasoning accuracy in general-purpose LLMs without relying on additional training data. We formalize the task following the structure of the FinQA benchmark (Chen et al., 2021).

3.1 TASK DEFINITION

The input of our task is a financial context $C = (T, X)$, where T denotes structured tables of data (e.g., income statements, balance sheets) and X denotes accompanying textual evidence (e.g., earnings call transcripts). Given a natural-language question q , the language model must produce a reasoning trace r and a final numerical answer y . Thus, the model implements a mapping

$$p_{\theta} : (C, q) \rightarrow (r, y), \quad (1)$$

where θ represents the model parameters. See appendix C for a full example of financial context, question, and expected answer in the FinQA benchmark.

3.2 EVALUATION METRICS

We evaluate models according to four criteria:

Answer accuracy. The final answer y must match the ground-truth answer y^* , up to differences in order of magnitude to account for unit differences such as 0.3B and 300M.

Format accuracy. The model must return a single final answer enclosed in `\boxed{ }`. Outputs with zero or multiple boxes are considered not to adhere to the format requirement.

Generation length. We measure the number of generated tokens to empirically illustrate any change in inference cost among different methods and models.

Inference time. We track the average inference latency per sample to demonstrate the differences in inference time among different methods when used in real-world high-volume applications.

3.3 MODELS

We conduct experiments on two 7B models that illustrate the current performance gap in financial numerical reasoning: general-purpose model Qwen-2.5-7B-Instruct (Yang et al., 2024a) and domain-specialized model Fin-R1 (Liu et al., 2025). We choose to use two 7B models due to computational constraints, as we only have reliable access to two 80GB A100 GPUs.

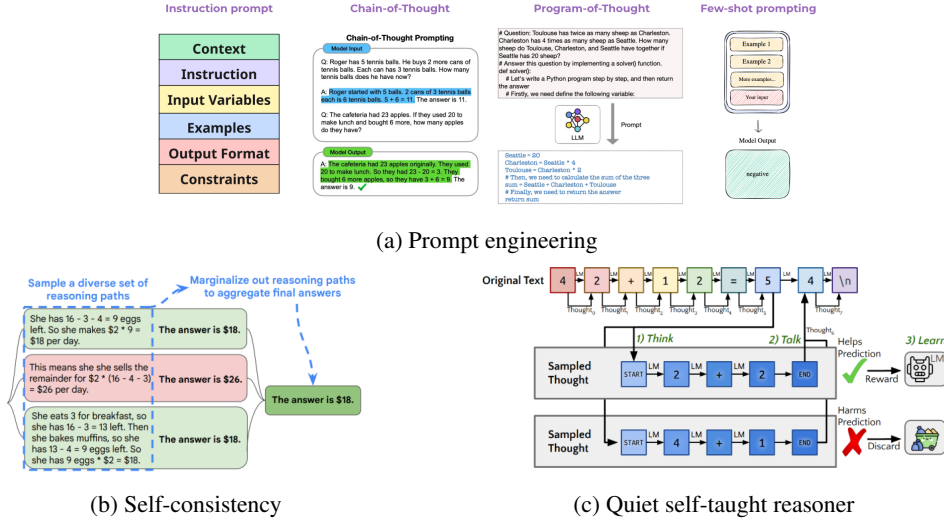


Figure 1: An overview of methods we study. (a) Four prompt families that we study for prompt engineering. (b) Illustration of self-consistency from Figure 1 in Wang et al. (2022). (c) Illustration of quiet self-taught reasoner from Figure 1 in Zelikman et al. (2024).

3.4 TARGET IMPROVEMENT

Our goal is to close a portion of the gap between Qwen-2.5-7B-Instruct and Fin-R1 by achieving at least **+4% absolute accuracy improvement** over the general-purpose baseline, where 4% is the reported difference between Fin-R1 and the best general-purpose model on FinQA (table 1).

4 METHODS

In this section, we provide an overview of the three methods we study and the specific implementation details that we use to adapt these methods to the financial numerical reasoning task. Figure 1 shows an illustrative overview of the methods we study.

4.1 PROMPT ENGINEERING

Prompt engineering is a simple user-end intervention for improving financial numerical reasoning. It guides the model to reason predictably and reliably using structured instructions that direct the model to perform specific reasoning steps, such as extracting values from input, showing multi-step calculations step-by-step, and formatting the final answer accurately. This makes prompt engineering the lightest-weight technique among the three approaches explored in this work to improve reasoning and accuracy in the financial domain.

Furthermore, many papers that establish state-of-the-art model, including Fin-R1 (Liu et al., 2025), do not release the specific prompts used during their evaluation. Consequently, their strong performance is difficult to reproduce, and the fairness and competitiveness of their baselines are difficult to verify. Our thorough prompt engineering study addresses this issue by systematically evaluating both the baseline model and the state-of-the-art model across a diverse range of prompt families and styles, establishing a fairer and competitive baseline.

We evaluate a combination of four families of prompts, each targeting a specific source of reasoning difficulty. **Instruction prompting** provides minimal task descriptions and formatting requirements, establishing a baseline with minimal structure. **Chain-of-Thought prompting** (Wei et al., 2022) adds explicit step-by-step guidance, either in free-form or using structured templates that require explicit value extraction, calculation, and output formatting. **Program-of-Thought prompting** (Chen et al., 2022) instructs the model to generate pseudo-code-style format resembling spreadsheet operations, offering greater precision in arithmetic. **Few-shot in-context learning** (Agarwal et al.,

2024) supplies two to three curated demonstrations illustrating clean extraction, concise multi-step calculations, and proper formatting.

Appendix A shows 10 prompts that we designed following these four prompt families. All the instructions are tailored to the financial numerical reasoning task. All the examples for few-shot prompting are written following the FinQA format. We use temperature = 0.7, to-p = 0.8, and max new generated tokens = 4000 throughout our experiments.

4.2 SELF-CONSISTENCY

Self-consistency decoding is an inference-time strategy that replaces standard greedy decoding or nucleus sampling for reasoning-intensive tasks (Wang et al., 2022). Given a model $p_\theta(r, y | C, q)$, traditional decoding methods approximate the solution by selecting a single most probable sequence, implicitly assuming that the highest-likelihood reasoning path yields the correct answer. In contrast, self-consistency seeks to approximate the marginal distribution over answers by sampling multiple reasoning paths and aggregating their outcomes.

Formally, self-consistency approximates the most consistent answer as

$$\hat{y} = \arg \max_y \sum_{r \in \mathcal{R}} p_\theta(y | r, C, q), \quad (2)$$

where $\mathcal{R}$ denotes a set of sampled reasoning paths. In practice, this marginalization is implemented by sampling N reasoning trajectories $\{r_i\}_{i=1}^N$ using stochastic decoding and selecting the answer that appears most frequently among the generated outputs.

Following Wang et al. (2022), self-consistency consists of the four steps:

Step 1: Sample multiple reasoning paths. Given an input (C, q) , we sample $N \in \{1, 3, 5, 9\}$ independent generations from the model. Each generation produces a reasoning path r_i and a corresponding final answer y_i . To control the diversity of sampled reasoning paths, we consider two sets of generation parameters: (1) conservative sampling with temperature = 0.7 and top-p = 0.8, and (2) high-variance sampling with temperature = 1.0 and top-p = 0.6. Higher variance settings encourage exploration of diverse reasoning trajectories, which can help mitigate single-path reasoning failures.

Step 2: Extract and normalize final answers. From each generated output, we extract the final answer by parsing the model’s boxed answer expression. To ensure that semantically equivalent answers are treated consistently during aggregation, we apply a normalization pipeline that removes currency symbols and formatting artifacts and normalizes the magnitude of the numerical answers. The last step of *magnitude normalization* allows numerically equivalent values expressed in different forms (e.g., 1.2, 120%, 0.012) to be mapped to a single canonical floating-point representation. This normalization step is essential for accurate majority voting in self-consistency.

We note that this normalization step is our own contribution to adapt self-consistency to the financial numerical reasoning domain.

Step 3: Aggregate answers via majority voting. Let $\{\tilde{y}_i\}_{i=1}^N$ denote the normalized answers. We compute the final prediction by applying unweighted majority voting:

$$\hat{y} = \arg \max_y \sum_{i=1}^N \mathbf{1}(\tilde{y}_i = y). \quad (3)$$

This aggregation marginalizes over reasoning paths and selects the most consistent answer, following the formulation proposed by Wang et al. (2022). We break ties by returning any answer that appears most frequently. In other words, $\hat{y}$ can be a collection of answers.

Step 4: Return consistent reasoning paths. Finally, we retain and return all reasoning paths r_i whose normalized final answers is in $\hat{y}$. These paths provide interpretable and diverse rationales supporting the final selected prediction.

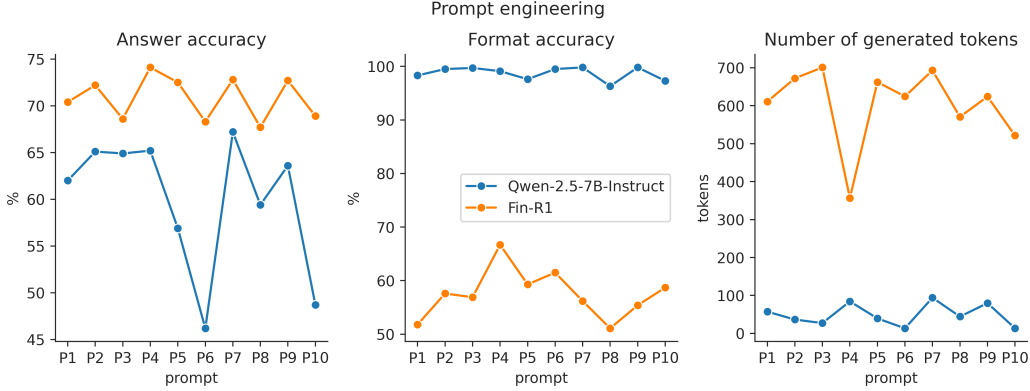


Figure 2: Answer accuracy, format accuracy, and average number of generated tokens when using different prompts on Qwen-2.5-7B-Instruct and Fin-R1. The blue lines are Qwen-2.5-7B-Instruct results. The orange lines are Fin-R1 model results. The left panel shows the answer accuracy in percentage. The middle panel shows the format accuracy in percentage. The right panel shows the average number of generated tokens per sample.

4.3 QUIET SELF-TAUGHT REASONER

Quiet self-taught reasoner is a data-free self-improvement method designed to enhance reasoning quality without additional chain-of-thought data (Zelikman et al., 2024). The method consists of three steps: Think, Talk, and Learn. **Think.** Across every single token in the input sequence, the model produces multiple reasoning traces in parallel, resulting in $n \times r \times t$ rationale matrix, where n is the number of tokens in the input sequence, r is the number of thoughts, and t is the length of thoughts. **Talk.** An MLP mixing head weighing the final hidden state of each of the $n \times r$ reasoning traces, combined with the base language model predicted logits, to return a single combined predicted logits for predicting the next token. **Learn.** The REINFORCE algorithm is used to optimize for the next token log loss by performing gradient updates on the base language model, the thought tokens, and the MLP mixing head.

We adapt the original Quiet self-taught reasoner codebase at <https://github.com/ezelikman/quiet-star>, originally written for Mistral models, to Qwen 2.5 models. Initial experiments, as reported in section 5.3, show that training with the original code imposes extreme memory requirements exceeding our compute budget of two 80GB A100 GPUs. To mitigate this issue, we (1) activate *gradient checkpointing* to reduce activation memory during fine-tuning, (2) fine-tune with *low-rank adapters* to reduce the number of trainable parameters from 7B to 10M, and (3) use *gradient accumulation* to achieve the same effective batch size of 8 as in Zelikman et al. (2024) without exceeding GPU memory limits.

We use the same hyperparameters as in Zelikman et al. (2024), including the number of thoughts, length of thoughts, number of training steps, warmup steps, learning rate, and weight decay coefficient. Appendix B provides a detailed description of our experiment setup for the quiet self-taught reasoner method.

5 EXPERIMENTS

5.1 PROMPT ENGINEERING

We begin by examining the performance gap between Fin-R1 and Qwen-2.5-7B-Instruct across various prompts. While the Fin-R1 paper reports a 16% absolute gap between the two models (76% vs 60%) (table 1), our prompt engineering results reveal a substantially smaller gap. Across ten prompts, Fin-R1 achieves a best accuracy of 74.1% with Prompt 4, whereas Qwen-2.5-7B-Instruct achieves a best accuracy of 67.2% with Prompt 7 (left panel, fig. 2 and table 4). This performance gap is therefore approximately 7%, **50% lower than the reported performance gap** in Liu et al.

(2025). This discrepancy highlights that prompt design significantly affects the evaluation of LLMs on the financial reasoning task, and a strong baseline cannot be established without a thorough prompt engineering study.

Surprisingly, there is a significant difference in format accuracy between the two models. While Qwen-2.5-7B-Instruct maintains 97-99% accuracy in producing exactly one final answer enclosed in `\boxed`, Fin-R1 format accuracy is consistently under 70% (middle panel, fig. 2 and table 5). Manual inspection shows that Fin-R1 frequently appends additional commentary or generates multiple boxed expressions, often due to overly long rationales. This behavior is undesirable in practice, as producing a clear and concise final answer is essential for end users to be able to extract the model’s predictions accurately.

Differences in generation length further support the observation that Fin-R1 produces overly lengthy reasoning traces. Qwen-2.5-7B-Instruct produces concise rationales, typically fewer than 100 tokens, whereas Fin-R1 often exceeds 1000 tokens per sample (right panel, fig. 2 and table 6). As a result, **Qwen-2.5-7B-Instruct’s inference time is 9 times faster on average** (table 7), despite both models having the same parameter count of 7B (table 1). This efficiency gap would be significant in practice, especially for large-scale real-time financial applications.

Together, these results show that prompt engineering establishes a stronger and fairer baseline for state-of-the-art financial numerical reasoning models, and Qwen-2.5-7B-Instruct exhibits significantly better format adherence and inference cost compared to Fin-R1.

5.2 SELF-CONSISTENCY

In this section, we evaluate whether self-consistency can improve Qwen-2.5-7B’s performance on FinQA. The results in fig. 3 show a clear and consistent benefit. The full table of results can be found at tables 8, 10 and 11. Using the best hyperparameter configuration, accuracy rises from 67.2% to 73.0%, **a +5.8% improvement**. This exceeds our 4% target and demonstrates that a general-purpose model can substantially benefit from self-consistency.

The left panel of fig. 3 also shows that the primary driver of improvement is the number of generated sequences N . Increasing number of sequences from 1 to 9 consistently increases accuracy, indicating that self-consistency can correct for single-generation errors produced by the Qwen-2.5-7B-Instruct model. Diversity parameters temperature and top-p also matter: slightly higher diversity (temperature = 1.0, top-p = 0.6) improves the change of sampling different reasoning traces and strengthens the majority vote signal, particularly with a higher number of generated sequences ($N \in \{5, 9\}$).

A potential concern with self-consistency is whether multiple generations per input would make inference too slow and costly. Surprisingly, **Qwen-2.5-7B-Instruct remains faster than Fin-R1** under every setting. Even with 9 samples, Qwen-2.5-7B-Instruct’s per-sample inference time is 33.7 seconds, far below Fin-R1 single-generation’s 49.6 seconds (middle panel, fig. 3). Two reasons explain this result: First, our prompt engineering results show that the number of generated tokens by Qwen-2.5-7B-Instruct is, on average, 9 times shorter than Fin-R1; thus, we can expect to be able to use self-consistency on Qwen-2.5-7B-Instruct with $N = 9$ with comparable inference time to Fin-R1. Second, while Fin-R1 has to generate tokens in its long single output sequentially, Qwen-2.5-7B-Instruct with self-consistency generates $N = 9$ outputs in parallel, further reducing its inference time.

Self-consistency naturally increases compute: with $N = 9$, Qwen-2.5-7B-Instruct generates 20% more total tokens than a single Fin-R1 generation, from 692.9 to 860.4 (right panel, fig. 3 and table 10). However, unlike Fin-R1, self-consistency requires no additional training data, avoiding the substantial upfront cost of collecting and curating expert-labeled financial chain-of-thought. This highlights a practical trade-off in improving financial numerical reasoning: Fin-R1 relies on expensive domain data and fine-tuning, whereas self-consistency increases inference-time compute but remains data-free.

Overall, these findings show that self-consistency is an effective inference-time method that can significantly improve accuracy while maintaining low latency and a small increase in inference cost.

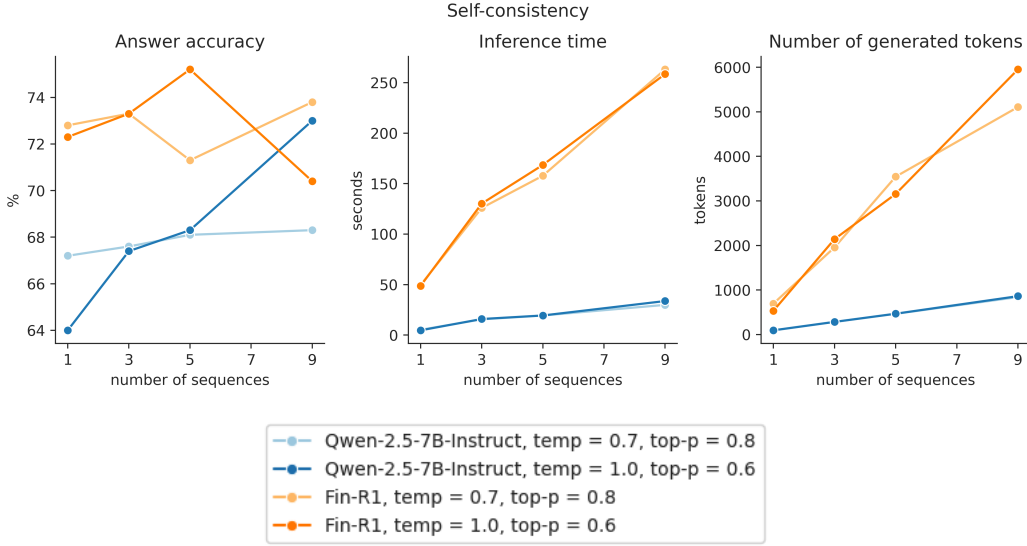


Figure 3: Answer accuracy, inference time, and number of generated tokens when applying self-consistency on Qwen-2.5-7B-Instruct and Fin-R1. The blue lines are Qwen-2.5-7B-Instruct results. The orange lines are Fin-R1 model results. The lighter lines are results with conservative sampling (temperature = 0.7, top-p = 0.8). The darker lines are results with high-variance sampling (temperature = 1.0 and top-p = 0.6). The left panel shows the answer accuracy in percentage. The middle panel shows inference time in seconds. The right panel shows the number of generated tokens per sample.

5.3 QUIET SELF-TAUGHT REASONER

Although Quiet self-taught reasoner is an appealing data-free fine-tuning approach to improve financial numerical reasoning, our experiments show that its resource requirement far exceeds our compute budget. Thus, in this section, we will discuss practical challenges that we encounter and our attempts to resolve them.

Memory scaling. The first challenge with this method is its need to cache several sequences of thoughts per token. In Zelikman et al. (2024), two parameters are used to determine the number of thoughts n and the length of thought t . The set of parameters that Zelikman et al. (2024) use is at least 8 thoughts of length 4. However, when adapting their code to Qwen-2.5-7B-Instruct, even with a small configuration (e.g., 2 thoughts of length 2), the amount of memory cached already reaches 117GB, requiring at least two 80GB A100 GPUs.

Table 2 shows the amount of memory cached across several small configurations. We observe that the amount of memory cached is proportional to the sum of the number of thoughts and the length of thoughts

$$\text{Memory cached} \propto \# \text{ Thoughts} + \text{Length of thoughts}, \quad (4)$$

meaning we would need at least 350GB, or 5 80GB A100 GPUS to run a configuration of 8 thoughts of length 4.

We attempted several mitigation strategies: gradient checkpointing, low-rank adapter, and gradient accumulation. These methods successfully reduced the memory allocated to 14GB and memory cached to 17-20GB even at the highest configuration (up to 24 thoughts of length 12), as well as reducing the number of trainable parameters from 7B to 10M. Table 12 shows that our strategies keep memory usage constant regardless of the number of thoughts and the length of thoughts.

Training time. Quiet self-taught reasoner also requires exceptionally long training time. Table 12 shows that the per-iteration time increases with the configuration size and ranges from half a minute to over two minutes. Zelikman et al. (2024) reportedly trains their models for 1000-4000 training

Table 2: Memory usage (in GB) when adapting the original quiet self-taught reasoner code from Zelikman et al. (2024) to Qwen-2.5-7B-Instruct.

# Thoughts	Length of thoughts	Memory allocated	Memory cached
1	1	51	62
1	2	51	91
2	1	51	91
2	2	51	117
4	2	51	176

Table 3: Per-iteration training time and per-sample inference time (in seconds) for quiet self-taught reasoner with Qwen-2.5-7B-Instruct.

# Thoughts	Length of thoughts	Training time	Inference time
8	4	38.7	162.4
10	4	45.3	189.2
12	4	52.7	219.1
16	8	84.0	351.5
24	12	138.7	580.3

steps, i.e., iterations. We attempt to use the same hyperparameters reported in Zelikman et al. (2024) and train Qwen-2.5-7B-Instruct with 8 thoughts of length 4, 10 thoughts of length 4, and 12 thoughts of length 4 for 1000 steps. Each model is trained for 12 hours, with checkpoints being saved every 100 training steps. However, the model only produces gibberish regardless of the configuration or the number of training steps. Figure 5 shows some examples of outputs that the model produces.

We notice that Zelikman et al. (2024) reports using eight 80GB H100 GPUs, which are 4 times more GPUs than our compute budget, each 3-4 times more powerful than A100 GPUs. Therefore, our current hypothesis for our training failure is that the quiet self-taught reasoner model requires significantly more training than anticipated, perhaps 10-100 epochs instead of 1000-4000 training steps. Given that 1000 training steps only correspond to 16% of an epoch under our dataset size of 6000, training for 10 epochs would take 30 days on our two 80GB A100 GPUs. Thus, full training was computationally infeasible with our computing budget.

Inference cost. Even if training has been successful, inference time presents another barrier. We limit the number of new generated tokens to 100 for this experiment. Table 3 shows that inference time for the quiet self-taught reasoner method grows with the number of thoughts and length of thoughts, ranging between 3 and 10 minutes. In contrast, the inference time of Fin-R1 is only 49.6 seconds, and the inference time of Qwen-2.5-7B-Instruct with self-consistency ($N = 9$) is only 33.7 seconds (table 11). These results imply that deployment of a quiet self-taught reasoner model would be prohibitively expensive even if training time were not a limiting factor.

6 CONCLUSION

This project studies whether financial numerical reasoning in large language models can be improved without additional training data. Using the FinQA benchmark, we evaluate three data-free approaches: prompt engineering, self-consistency, and quiet self-taught reasoner - on the general-purpose model Qwen-2.5-7B-Instruct. Our findings highlight several key insights.

First, although Fin-R1 remains the state-of-the-art model for financial numerical reasoning, we show that prompt engineering is essential for establishing fair and competitive baselines. Carefully structured few-shot prompts substantially narrow the performance gap between Fin-R1 and general-purpose models such as Qwen-2.5-7B-Instruct from 16% to 7%, while also improving formatting stability by 50% and inference efficiency by 9 times.

Second, **self-consistency improves accuracy by +5.8% on Qwen-2.5-7B-Instruct** - surpassing our 4% target - while remaining faster per sample than Fin-R1 and only generates 20% more tokens.

This demonstrates that self-consistency is a simple inference-time strategy that can significantly enhance reasoning ability of LLMs in financial tasks.

Finally, quiet self-taught reasoner is a promising direction for data-free fine-tuning, but our analysis shows that its practical compute requirements are prohibitively expensive. Extreme memory scaling, slow forward passes, and multi-minute per-sample inference times make the method currently impractical for modest hardware budgets, despite its conceptual appeal.

Overall, our study demonstrates that substantial gains in financial numerical reasoning are achievable at inference time alone, but that more scalable training-time methods are needed to achieve state-of-the-art performance without additional training data.

Future work. Our findings open several avenues for future study. For self-consistency, a dynamic inference policy could reduce cost by switching between single-pass decoding and self-consistency based on estimated model uncertainty (e.g., values of next token logits). In addition, a more comprehensive hyperparameter exploration may yield further accuracy gains without increasing compute. To make quiet self-taught reasoner viable, new architectures are needed to parallelize thought generations in order to reduce inference time. In addition, access to more powerful accelerators (e.g., H100 clusters) would allow us to perform a full training run to quantify the true performance potential of the method.

AUTHOR CONTRIBUTIONS

Smruti Nalawad conducted experiments, provided data, conducted analysis for sections 4.1 and 5.1 and wrote the manuscript for these sections. Haitong Zhu conducted experiments, provided data, conducted analysis for sections 4.2 and 5.2, wrote the manuscript for these sections and section 2, and set up the initial repository for the project. Nhi Nguyen conducted experiments, provided data, conducted analysis for sections 4.3 and 5.3, wrote the manuscript for these sections, sections 1, 3 and 6, appendix and abstract. All authors reviewed the final manuscript.

REFERENCES

- Rishabh Agarwal, Avi Singh, Lei Zhang, Bernd Bohnet, Luis Rosias, Stephanie Chan, Biao Zhang, Anshu Anand, Zaheer Abbas, Azade Nova, et al. Many-shot in-context learning. *Advances in Neural Information Processing Systems*, 37:76930–76966, 2024.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.
- Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan R Routledge, et al. Finqa: A dataset of numerical reasoning over financial data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 3697–3711, 2021.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213, 2022.
- Xiao-Yang Liu, Guoxuan Wang, Hongyang Yang, and Daochen Zha. Fingpt: Democratizing internet-scale data for financial large language models. *arXiv preprint arXiv:2307.10485*, 2023.
- Zhaowei Liu, Xin Guo, Fangqi Lou, Lingfeng Zeng, Jinyi Niu, Zixuan Wang, Jiajie Xu, Weige Cai, Ziwei Yang, Xueqian Zhao, et al. Fin-r1: A large language model for financial reasoning through reinforcement learning. *arXiv preprint arXiv:2503.16252*, 2025.
- Sedigheh Mahdavi, Pradeep Kumar Joshi, Lina Huertas Guativa, Upmanyu Singh, et al. Integrating large language models in financial investments and market analysis: A survey. *arXiv preprint arXiv:2507.01990*, 2025.
- Pragya Srivastava, Manuj Malik, Vivek Gupta, Tanuja Ganu, and Dan Roth. Evaluating llms’ mathematical reasoning in financial document question answering. *arXiv preprint arXiv:2402.11194*, 2024.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhajan Kambadur, David Rosenberg, and Gideon Mann. Bloomberggpt: A large language model for finance. *arXiv preprint arXiv:2303.17564*, 2023.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024a.

Xinqi Yang, Scott Zang, Yong Ren, Dingjie Peng, and Zheng Wen. Evaluating large language models on financial report summarization: An empirical study. *arXiv preprint arXiv:2411.06852*, 2024b.

Xuan Yao, Qianteng Wang, Xinbo Liu, and Ke-Wei Huang. Evaluating large language models for financial reasoning: A cfa-based benchmark study. *arXiv preprint arXiv:2509.04468*, 2025.

Eric Zelikman, Yuhuai Wu, and Noah D Goodman. Star: Self-taught reasoner. In *Proceedings of the NIPS*, volume 22, 2022.

Eric Zelikman, Georges Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah D Goodman. Quiet-star: Language models can teach themselves to think before speaking. *arXiv preprint arXiv:2403.09629*, 2024.

A PROMPTS

P1. General instructions

```
SYSTEM_PROMPT:
You are a financial reasoning assistant.
Think step-by-step inside <think>...</think>.
Use the given financial context (text + tables) to extract numbers and
perform calculations.
After reasoning, output ONLY the final numeric answer inside a single
LaTeX box.}

TASK_PROMPT:
Use the context to identify relevant values and compute the correct final
answer.

ANSWER_FORMAT:
<think>
(your reasoning steps here)
</think>
<answer>
\boxed{FINAL_ANSWER}
</answer>
```

P2. Persona, Task, Context, and Format

SYSTEM_PROMPT:

You are a financial reasoning assistant specialized in extracting numerical information from financial text and tables and performing precise calculations.
Carefully analyze the provided financial context, identify all relevant values, perform step-by-step reasoning, and compute the correct final numeric result. Follow best practices: be precise, avoid ambiguity, and output one boxed final value.
Think step-by-step, showing clear intermediate reasoning in <think>. Use only the numbers and information present in the given financial text and tables. All reasoning must be performed internally within the <think>...</think> block.

TASK PROMPT:

Use the provided financial narrative and table to extract the correct values, perform multi-step reasoning, and compute the final numeric answer.
Be careful: the context may contain distractor numbers or irrelevant lines.

ANSWER_FORMAT:

```
<think>
(show all reasoning steps and calculations)
</think>
<answer>
\boxed{FINAL_ANSWER}
</answer>
```

P3. General Chain-of-Thought prompting

SYSTEM_PROMPT:

You are a financial reasoning assistant. Follow this decomposition:
1) Break the problem into subparts.
2) Solve each subpart inside <think>.
3) Combine results into one final numeric answer in a box.

TASK PROMPT:

Break the question -> solve components -> combine for final answer.

ANSWER_FORMAT:

```
<think>
Breakdown: ...
Sub-solutions: ...
Combine: ...
</think>
<answer>
\boxed{FINAL_ANSWER}
</answer>
```

P4. Templated-based Chain-of-Thought prompting

SYSTEM_PROMPT:

You are a financial reasoning assistant. Follow these rules exactly:
1. Read the context carefully and identify all numeric values.
2. Determine which values are relevant to the question.
3. Show reasoning inside <think>...</think> in numbered steps.
4. Perform calculations sequentially, stating intermediate values.
5. Output only one numeric answer inside a single LaTeX box.
6. Do NOT include any text after the boxed answer.

TASK PROMPT:

Use the rules above to extract required values and compute the final numeric answer.

ANSWER_FORMAT:

```
<think>
Step 1: (identify numbers)
Step 2: (set formula)
Step 3: (compute sequentially)
</think>
<answer>
\boxed{FINAL_ANSWER}
</answer>
```

P5. Rule-based Chain-of-Thought prompting

SYSTEM_PROMPT:
You are a financial reasoning assistant. Think step-by-step inside <think>...</think>. You MUST NOT output any text outside the <think> block except for one final boxed numeric answer. Do not explain, justify, or add any words after the boxed answer.

TASK_PROMPT:
Use the context to extract the required numbers, compute the correct value, and follow strict output rules.

ANSWER_FORMAT:
<think>
(step-by-step reasoning here)
</think>
<answer>
\boxed{FINAL_ANSWER}
</answer>

P6. General Program-of-Thought prompting

SYSTEM_PROMPT:
You are a financial reasoning assistant. You must analyze the financial context and extract a sequence of explicit mathematical operations. Always think inside <think>...</think> using the following rules:
1. Identify ALL relevant numbers from the text and tables.
2. Represent intermediate steps using explicit operators:
ADD(a,b), SUB(a,b), MUL(a,b), DIV(a,b), MAX(), MIN(), AVG(), TBL_SUM(), DIFF(a,b), RATIO(a,b), TABLE_LOOKUP(row,col).
3. Show each operation in order inside <think>.
4. After completing the reasoning, compute the final numeric value.
5. Output ONLY the final numeric result with two-decimal precision inside \boxed{ }.
Do NOT output words, sentences, or explanations in the final answer.

TASK_PROMPT:
Extract all relevant financial values, express intermediate steps using program-style operations, compute the final result, and return ONLY the final numeric output.

ANSWER_FORMAT:
<think>
Extract numbers -> write program-style operations (ADD(), SUB(), MUL(), DIV(), etc.) -> compute result.
</think>
<answer>
\boxed{FINAL_NUMERIC_ANSWER}
</answer>

P7. Few-shot In-context learning

SYSTEM_PROMPT:
You are a financial reasoning assistant.
Think step-by-step inside <think>...</think>.
Use the given financial context (text + tables) to extract numbers and perform calculations.

After reasoning, output ONLY the final numeric answer inside a single LaTeX box.
Following the simple examples below.

Example 1:

Context: The company reports revenue of 200 million dollars and costs of 50 million dollars.

Given the context, calculate the profit.

```
<think>Profit = Revenue - Cost = 200 - 50 = 150 (million dollars)</think>
<answer>\boxed{150}</answer>
```

Example 2:

Context: The asset's price rises from 80 to 100 thousand dollars over the period.

Given the context, how much did the asset's price increase?

```
<think>Increase = New Price - Old Price = 100 - 80 = 20</think>
<answer>\boxed{20}</answer>
```

TASK PROMPT:

Use the reasoning style shown in the examples to compute the correct numeric answer using the given financial context.

ANSWER_FORMAT:

```
<think>
(your reasoning following the example style)
</think>
<answer>
\boxed{FINAL_ANSWER}
</answer>
```

P8. Few-shot In-context learning with Table reasoning

SYSTEM_PROMPT:

You are a financial reasoning assistant.

Think step-by-step inside <think>...</think>.

Use the given financial context (text + tables) to extract numbers and perform calculations.

After reasoning, output ONLY the final numeric answer inside a single LaTeX box.

Following the simple examples below on how to extract information from tables.

Example 1:

Context: Table:

For 2020, the Sales is 100.

For 2021, the Sales is 130.

Given the context, what is the increase in sales from 2020 to 2021?

```
<think>Sales in 2020 = 100; Sales in 2021 = 130 -> Increase = 130 - 100 = 30</think>
<answer>\boxed{30}</answer>
```

Example 2:

Context: Table:

For Q1, the Revenue is 200.

For Q2, the Revenue is 250.

Given the context, what is the total revenue for the first two quarters?

```
<think>Revenue Q1 = 200; Revenue Q2 = 250 -> Total = 200 + 250 = 450</think>
<answer>\boxed{450}</answer>
```

TASK PROMPT:

Use the example pattern to extract table values and compute the correct answer.

ANSWER_FORMAT:

```
<think>
(extract table values -> compute difference/ratio/etc.)
</think>
<answer>
\boxed{FINAL_ANSWER}
</answer>
```

P9. Few-shot In-context learning with increasing difficulty

SYSTEM_PROMPT:
 You are a financial reasoning assistant.
 Think step-by-step inside <think>...</think>.
 Use the given financial context (text + tables) to extract numbers and
 perform calculations.
 After reasoning, output ONLY the final numeric answer inside a single
 LaTeX box.
 Following the simple examples below.

Example 1 (easy)

Context: A company reports that its operating revenue is 120 and its
 operating expenses are 45.

Given the context, what is the operating profit?

```
<think> Profit = 120 - 45 = 75</think>
<answer>\boxed{75}</answer>
```

Example 2 (medium)

Context: A fund initially holds 80 units of an asset. It purchases 20
 additional units. Each unit has a market price of 6.

Given the context, what is the total market value of the fund's holdings?

```
<think>
A = ADD(80, 20) = 100
B = MUL(100, 6) = 600
</think>
<answer>\boxed{600}</answer>
```

Example 3 (hard)

Context: A company generates quarterly revenues of:

Q1: 150

Q2: 180

Q3: 170

In Q4, revenue increases by 10% relative to Q3.

Given the context, what is the total annual revenue?

```
<think>
Q4 = MUL(170, 1.10) = 187
Total = ADD(150, 180) = 330
Total = ADD(330, 170) = 500
Total = ADD(500, 187) = 687
</think>
<answer>\boxed{687}</answer>
```

TASK PROMPT:

Use the reasoning style shown in the examples to compute the correct
 numeric answer using the given financial context.

ANSWER_FORMAT:

```
<think>
(your reasoning following the example style)
</think>
<answer>
\boxed{FINAL_ANSWER}
</answer>
```

P10. Few-shot Program-of-Thought

SYSTEM_PROMPT:

You are a financial reasoning assistant. You must analyze the financial context and extract a sequence of explicit mathematical operations. Always think inside `<think>...</think>` using the following rules:

1. Identify ALL relevant numbers from the text and tables.
2. Represent intermediate steps using explicit operators:
`ADD(a,b)`, `SUB(a,b)`, `MUL(a,b)`, `DIV(a,b)`, `MAX()`, `MIN()`, `AVG()`, `TBL_SUM()`, `DIFF(a,b)`, `RATIO(a,b)`, `TABLE_LOOKUP(row,col)`.
3. Show each operation in order inside `<think>`.
4. After completing the reasoning, compute the final numeric value.
5. Output ONLY the final numeric result with two-decimal precision inside `\boxed{}`.

Do NOT output words, sentences, or explanations in the final answer. Following the simple examples below.

Example 1:

Context: A product has a base cost of 50. It also requires an additional fee of 20.

After adding the fee, the company applies a markup factor of 2x to determine the final price.

Given the context, what is the final price after adding the fee and applying the markup?

```
<think>
COST= ADD(50, 20) = 70
PRICE = MUL(70, 2) = 140
</think>
<answer>\boxed{140}</answer>
```

Example 2:

Context: A company sells a subscription service with a monthly base rate of 30. Each customer also pays a maintenance surcharge of 10.

At the end of the month, the company applies a 3x multiplier to estimate the total projected quarterly revenue per customer.

Given the context, what is the projected quarterly revenue per customer?

```
<think>
MONTHLY = ADD(30, 10) = 40
QUARTER = MUL(40, 3) = 120
</think>
<answer>\boxed{120}</answer>
```

TASK PROMPT:

Extract all relevant financial values, express intermediate steps using program-style operations, compute the final result, and return ONLY the final numeric output.

ANSWER_FORMAT:

```
<think>
Extract numbers -> write program-style operations (ADD(), SUB(), MUL(),
    DIV(), etc.) -> compute result.
</think>
<answer>
\boxed{FINAL_NUMERIC_ANSWER}
</answer>
```

B QUIET SELF-TAUGHT REASONER IMPLEMENTATION DETAILS

Following Zelikman et al. (2024), we use the AdamW optimizer with a warmup of 20 steps, a learning rate of 1e-6, a weight decay of 0.001, and an effective batch size of 8. We train the model for 1000 training steps, which corresponds to 16% of a full epoch.

Explore FinQA

devon energy corporation and subsidiaries notes to consolidated financial statements 2013 (continued) debt maturities as of december 31 , 2012 , excluding premiums and discounts , are as follows (in millions) : .

2013	\$ 3189
2014	500
2015	2014
2016	500
2017	750
2018 and thereafter	675
total	\$ 11664

credit lines devon has a \$ 3.0 billion syndicated , unsecured revolving line of credit (the 2010senior credit facility 2014) . the senior credit facility has an initial maturity date of october 24 , 2017 . however , prior to the maturity date , devon has the option to extend the maturity for up to two additional one-year periods , subject to the approval of the lenders . amounts borrowed under the senior credit facility may , at the election of devon , bear interest at various fixed rate options for periods of up to twelve months . such rates are generally less than the prime rate , however , devon may elect to borrow at the prime rate . the senior credit facility currently provides for an annual facility fee of \$ 3.8 million that is payable quarterly in arrears . as of december 31 , 2012 , there were no borrowings under the senior credit facility . the senior credit facility contains only one material financial covenant . this covenant requires devon 2019s ratio of total funded debt to total capitalization , as defined in the credit agreement , to be no greater than 65 percent . the credit agreement contains definitions of total funded debt and total capitalization that include adjustments to the respective amounts reported in the accompanying financial statements . also , total capitalization is adjusted to add back noncash financial write-downs such as full cost ceiling impairments or goodwill impairments . as of december 31 , 2012 , devon was in compliance with this covenant with a debt-to- capitalization ratio of 25.4 percent . commercial paper devon has access to \$ 5.0 billion of short-term credit under its commercial paper program . commercial paper debt generally has a maturity of between 1 and 90 days , although it can have a maturity of up to 365 days , and bears interest at rates agreed to at the time of the borrowing . the interest rate is generally based on a standard index such as the federal funds rate , libor , or the money market rate as found in the commercial paper market . as of december 31 , 2012 , devon 2019s weighted average borrowing rate on its commercial paper borrowings was 0.37 percent . other debentures and notes following are descriptions of the various other debentures and notes outstanding at december 31 , 2012 , as listed in the table presented at the beginning of this note .

Example id : DVN/2012/page_77.pdf-3

Question : as of december 31 , 2012 what was the ratio of the devon debt maturities in 2013 compared to 2014

Program : divide(3189, 500)

Answer : 6.378

[Prev](#) [Next](#)

Figure 4: An example in FinQA data from the webpage <https://finqasite.github.io/explore.html>

C FINQA EXAMPLE

D FULL TABLES OF RESULTS

Table 4: Accuracy in percentage with different prompts. Best results are highlighted in bold.

Prompt	Qwen-2.5-7B-Instruct	Fin-R1
P1 - General instructions	62.0	70.4
P2 - Persona, Task, Context, and Format	65.1	72.2
P3 - General CoT prompting	64.9	68.6
P4 - Templated-based CoT prompting	65.2	74.1
P5 - Rule-based CoT prompting	56.9	72.5
P6 - General PoT	46.2	68.3
P7 - Few-shot	67.2	72.8
P8 - Few-shot with Table reasoning	59.4	67.7
P9 - Few-shot with increasing difficulty	63.6	72.7
P10 - Few-shot PoT	48.7	68.9

Table 5: Format accuracy in percentage with different prompts.

Prompt	Qwen-2.5-7B-Instruct	Fin-R1
P1 - General instructions	98.3	51.8
P2 - Persona, Task, Context, and Format	99.5	57.6
P3 - General CoT prompting	99.7	56.9
P4 - Templated-based CoT prompting	99.1	66.7
P5 - Rule-based CoT prompting	97.6	59.3
P6 - General PoT	99.5	61.5
P7 - Few-shot	99.8	56.2
P8 - Few-shot with Table reasoning	96.3	51.1
P9 - Few-shot with increasing difficulty	99.8	55.4
P10 - Few-shot PoT	97.3	58.7

Table 6: Total number of generated tokens with different prompts.

Prompt	Qwen-2.5-7B-Instruct	Fin-R1
P1 - General instructions	57.3	610.9
P2 - Persona, Task, Context, and Format	36.5	671.9
P3 - General CoT prompting	27.2	700.7
P4 - Templated-based CoT prompting	84.3	356.5
P5 - Rule-based CoT prompting	39.5	662.1
P6 - General PoT	13.4	624.3
P7 - Few-shot	94.1	692.9
P8 - Few-shot with Table reasoning	44.3	570.4
P9 - Few-shot with increasing difficulty	79.8	624.0
P10 - Few-shot PoT	13.6	521.3

Table 7: Inference time in seconds with different prompts.

Prompt	Qwen-2.5-7B-Instruct	Fin-R1
P1 - General instructions	6.6	50.9
P2 - Persona, Task, Context, and Format	8.5	58.4
P3 - General CoT prompting	8.8	51.5
P4 - Templated-based CoT prompting	7.2	27.3
P5 - Rule-based CoT prompting	4.7	47.6
P6 - General PoT	2.4	50.2
P7 - Few-shot	5.2	49.6
P8 - Few-shot with Table reasoning	5.8	50.9
P9 - Few-shot with increasing difficulty	6.5	47.6
P10 - Few-shot PoT	3.9	50.6

Table 8: Accuracy in percentage with Self-consistency. Best results are highlighted in bold.

# sequences	Temperature	Top-p	Qwen-2.5-7B-Instruct	Fin-R1
1	0.7	0.8	67.2	72.8
1	1.0	0.6	64.0	72.3
3	0.7	0.8	67.6	73.3
3	1.0	0.6	67.4	73.3
5	0.7	0.8	68.1	71.3
5	1.0	0.6	68.3	75.2
9	0.7	0.8	68.3	73.8
9	1.0	0.6	73.0	70.4

Table 9: Format accuracy in percentage with Self-consistency.

# sequences	Temperature	Top-p	Qwen-2.5-7B-Instruct	Fin-R1
1	0.7	0.8	99.8	56.2
1	1.0	0.6	99.7	56.4
3	0.7	0.8	99.9	53.6
3	1.0	0.6	99.9	55.6
5	0.7	0.8	99.8	57.8
5	1.0	0.6	99.8	55.2
9	0.7	0.8	99.7	54.0
9	1.0	0.6	99.7	60.9

Table 10: Total number of generated tokens with Self-consistency.

# sequences	Temperature	Top-p	Qwen-2.5-7B-Instruct	Fin-R1
1	0.7	0.8	94.1	692.9
1	1.0	0.6	93.4	531.7
3	0.7	0.8	282.9	1956.6
3	1.0	0.6	282.6	2141.7
5	0.7	0.8	464.0	3544.0
5	1.0	0.6	466.5	3156.0
9	0.7	0.8	840.6	5107.5
9	1.0	0.6	860.4	5952.6

Table 11: Inference time in seconds with Self-consistency.

# sequences	Temperature	Top-p	Qwen-2.5-7B-Instruct	Fin-R1
1	0.7	0.8	5.2	49.6
1	1.0	0.6	4.7	48.7
3	0.7	0.8	15.6	125.9
3	1.0	0.6	15.9	130.2
5	0.7	0.8	19.4	157.8
5	1.0	0.6	19.4	168.5
9	0.7	0.8	29.9	263.1
9	1.0	0.6	33.7	258.5

Table 12: Memory usage (in GB) and per-iteration time (in seconds) when employing gradient checkpointing, low-rank adapters, and gradient accumulation on quiet self-taught reasoner with Qwen-2.5-7B-Instruct.

# Thoughts	Length of thoughts	Memory allocated	Memory cached
8	4	14	17
10	4	14	17
12	4	14	17
16	8	14	17
24	12	14	20

Output 1:
 ____, 对 1在: ((\n (11,的, \n , \n(((: 1, 的1 ____ 的 (, , , 对 ____ 1
 (\n, \n(, ((1(, ____, ((: \n 1 , , ____ (\n ((, \n1 (, 1 (((,

Output 2:
 (, 的: (1 ,对 (11 1 :11:: : (____ ((, 1 , \n (: (\n ____ 在 , , ____ (, ((,
 ____:, , \n, :: (, 对:对 \n, \n (, (____: ____ , (1 ____ \n, (, \n :

Output 3:
 1 : \n (\n \n ____, , (\n, (: ____ 对 \n: (, 的 \n, :: ____, (, ____ :: : \n (\n:1 \n (____ (\n (: (____ (:, ((对 , ((____ , \n ____ 在的 ____: , ,

Output 4
 (____ (: , \n ____ (1 ____ (的 (:1 \n \n \n (____ , \n , \n \n, : \n (____ \n ____ : \n (____ , \n (: \n ____ (____ (在1的, :, ((, ____ \n 1 \n (, \n ____ 1 , 对1 (, \n11:1

Output 5
 (1 \n ((, (\n 的(对 (((: ____ (的 (: (: \n (, 在: 1 ((____ (\n, (:, ((, , \n ,对 \n (\n \n, ,: , , 在 (对 ____ \n,对, 对的, 的 (:, 的 ((在 \n (

Figure 5: Example outputs when train Qwen-2.5-7B-Instruct with quiet self-taught reasoner